# RCA Causal Taxonomy Metamodel

**Session**: April 25, 2026
**Purpose**: Define the taxonomy of causal categories that must be covered by an automated RCA pipeline for power plant equipment. This metamodel serves as a required coverage checklist — for a given event, the pipeline must either generate and score at least one candidate in each category, or explicitly document why the category was ruled out.

## Key Questions an Automated RCA Workflow Must Answer

### Scoping

- What failed, on which equipment, under what operating conditions?
- Which safety functions were challenged or lost?
- What is the investigation boundary — systems, trains, time window?

### Data and Relations

- What signals were anomalous before the event, and in what order?
- What maintenance, tests, or configuration changes occurred in the precursor window?
- Which anomalies are causes versus consequences?
- Are any relevant data sources missing, degraded, or of insufficient resolution?

### Pattern Recognition

- Is there a degradation trend preceding the event, and over what timescale?
- Is this a first occurrence or a recurrence? If recurrence, did prior corrective actions work?
- Is there evidence of common cause failure across multiple trains or components?
- Does the anomaly signature match a known failure mode?

### Hypothesis Generation and Ranking

- What are the plausible root causes, ranked by evidence strength?

- Which causal categories have been ruled out, and why?
- Are there near-tie hypotheses the evidence cannot discriminate between?
- Is the top-ranked hypothesis physically consistent with the operating conditions at the time?

## Conclusion

- What corrective actions address the proximate, contributing, and root cause levels?
- What barriers failed, and why?
- Was human performance a cause or contributor?
- What gaps in the investigation remain unresolved?
- What should be monitored to verify corrective action effectiveness?

# Causal Categories

## A. Equipment-Internal

*Component fails due to internal degradation mechanisms. Covered by standard FMEA. Primary source of KG failure mode nodes.*

- Material degradation
- Mechanical degradation
- Electrical degradation
- Instrumentation degradation
- Control component degradation

## B. Required Support Not Available or Degraded

*Equipment receives insufficient or absent support from ancillary systems. Requires connectivity graph reasoning: "if required support S is degraded AND equipment E depends on S, generate a candidate linking S degradation to E failure." Not derivable from FMEA alone.*

- Electrical power
- Cooling (process)
- Thermal management (heat tracing, HVAC, cooling to instrument rooms)
- Lubrication
- Sealing
- Instrument air

- Control signal
- Communications

## C. Upstream Influence

*Process conditions arriving at the equipment inlet are outside design basis. Requires fluid/energy path directionality — topology proximity alone is insufficient.*

- Insufficient inlet flow
- Poor fluid quality
- Entrained gas
- High inlet temperature
- Low suction pressure
- Wrong feed composition

## D. Downstream Influence

*Conditions imposed on the equipment outlet by downstream systems. Topology expansion captures neighboring components but does not reason about flow path direction.*

- Excessive backpressure
- Blocked discharge path
- Unstable demand
- Downstream isolation
- Induced recirculation

## E. Operating Context / Mission Demand

*Equipment is operated outside its design envelope or in a manner that accelerates degradation.* `operational_context.operating_point` *is collected by the pipeline but consumed by no scoring stage — this entire category is currently ignored.*

- Overload
- Off-design operation
- Thermal transients (load follow, heatup/cooldown rates)
- Intermittent cycling
- Start-stop stress
- Prolonged standby
- Runout / low-flow operation

## F. External Hazards and Disturbances

*Conditions imposed from outside the plant process boundary. No representation in current pipeline data model.*

- Thermal environment
- Flooding
- Seismic event
- Fire
- EMI / electrical disturbance
- Foreign object debris

## G. Human and Organizational Contributors

*Actions or omissions by individuals that directly caused or enabled the failure. Only hook in current pipeline is Ishikawa `maintenance_human_factors` branch (keyword-driven) and proposed WO-date proximity check (not yet implemented).*

> **Boundary note with Category I**: Category G covers human actions that deviated from a correct configuration baseline (wrong execution). Category I covers cases where the human action was correctly executed but the configuration baseline itself was wrong. A maintenance technician installing the wrong part when the procedure specified the right part is Category G. The same technician correctly installing a part specified incorrectly in an unrevised procedure is Category I.

- Operator misalignment
- Maintenance error
- Calibration error
- Wrong procedure used or procedure not followed
- Inadequate pre-job briefing
- Failure to use procedure
- Delayed response
- Incorrect setpoint

## H. Design and Specification Deficiencies

*Equipment performs as designed — the design itself is inadequate for the service conditions. FMEA almost never captures this because it assumes design adequacy. Distinct from Category A.*

- Undersized for actual duty
- Inadequate design margin

- Material specification wrong for service conditions
- Incompatible materials (galvanic, chemical)
- Thermal expansion not accommodated
- Fatigue life underestimated at design
- Vibration not analyzed at design

## I. Configuration and Change Control

*The human action was correctly executed but the configuration baseline was wrong or the change control process failed. Distinct from Category G (human contributors) — the deviation is from the design basis, not from procedure.*

- Unauthorized or undocumented modification
- Drawing or procedure not updated after change
- Temporary configuration not restored
- Setpoint change without engineering review
- Software or firmware change introduced defect
- Spare part substitution with non-equivalent component

## J. Inspection and Testing Program Inadequacy

*Equipment degraded between inspections not because maintenance was skipped but because the surveillance program was not designed to detect this failure mode at its actual progression rate. Connects to S-13 (prior CA ineffective) and S-11 (AMP gap).*

- Surveillance test interval too long to detect degradation
- Test methodology doesn't reveal actual failure mode
- Acceptance criteria not conservative enough
- Test performed under non-representative conditions
- Inspection technique lacks sensitivity for this degradation mechanism

## K. Vendor and Supply Chain

*Specification was correct but delivered item did not meet it, or a batch-level defect affects multiple installed components. Distinct from design deficiency (H) and maintenance error (G). Requires supply chain data (lot numbers, vendor certifications, receipt inspection records) with no current representation in pipeline.*

- Manufacturing defect in delivered component
- Wrong material certified to correct specification (material traceability failure)

- Batch-level defect affecting multiple installed components
- Vendor procedure deviation not disclosed
- Counterfeit or non-conforming part

## L. Systemic and Latent Organizational Weaknesses

*The root cause behind the root cause — the organizational system that allowed contributing causes (G through K) to persist. Requires qualitative evidence across program documents, trend data, and institutional knowledge. Hardest category to automate.*

- Corrective action program not effective (repeated escapes)
- Operating experience not incorporated
- Training program gap
- Resource or staffing constraint affecting maintenance quality
- Safety culture indicator

# Functional Architecture

## Design Principles

- Data and decisions/assessments are the two poles of the framework. Every reasoning step consumes defined data inputs and produces defined output artifacts — either intermediate assessments or final decisions.
- Human-in-the-loop decision points are explicit functional elements, not boundary conditions.
- Both primary output artifacts are traced throughout: the **hypothesis ranking** (scored candidate list) and the **RCA card** (structured conclusion for CAP).

## Data Elements

Organized by the reasoning step they primarily support. A data element may serve multiple steps.

## Structured / Model Data

| Data Element | Primary step | Causal categories | Schema coverage |
|---|---|---|---|
| Equipment hierarchy and topology (MBSE model) | Scoping, Relations | A–D, K | `kg_context.json` |
| Safety function definitions | Scoping, Conclusion | All | `kg_context.json` |
| Protection logic (trips, permissives, interlocks) | Relations, Patterns | A, B, F | `protection_logic_context.json` |
| Configuration change records (ECNs, setpoint changes) | Patterns, Candidates | H, I | `configuration_change_records.json` |

## Time-Series and Event Data

| Data Element | Primary step | Causal categories | Schema coverage |
|---|---|---|---|
| Telemetry (process historian: pressure, temperature, flow, vibration, electrical) | Patterns | A–F | `telemetry_summary.json` |
| Sequence of Events (SOE) recorder | Relations, Patterns | A, B, F | `soe_log.json` |
| Alarm logs | Relations, Patterns | A–F | `alarm_log.json` |
| Environmental monitoring (ambient temperature, humidity, grid disturbances, seismic) | Patterns | F | `environmental_monitoring.json` |

> **Note**: interpreting SOE records requires protection logic documentation (trip setpoints, permissive logic diagrams) as context — these two data elements are tightly coupled and must be available together.

## Maintenance and Operations Data

| Data Element | Primary step | Causal categories | Schema coverage |
|---|---|---|---|
| Work orders (corrective and preventive) | Patterns, Candidates | G, I, J | `cmms_context.json` (also `document.json`) |
| Surveillance and calibration records (as-found / as-left) | Patterns, Candidates | A, J | `pm_compliance.json` (also `document.json`) |
| Condition reports (CRs) | Patterns, Candidates | All | `cmms_context.json` (also `document.json`) |
| Operator logs and shift narratives | Relations, Patterns, Candidates | E, G | `operational_context.json` (also `document.json`) |

## Document and Institutional Knowledge

| Data Element | Primary step | Causal categories | Schema coverage |
|---|---|---|---|
| FMEAs | Candidates | A–E | `document.json` (ingestion: `fmea_ingestion_report.json`) |
| SOPs and procedures | Candidates, Conclusion | G, I | `document.json` |
| ECAs and RCAs | Candidates, Conclusion | All | `document.json` |
| Industry OE documents (NRC, INPO, EPRI) | Candidates, Conclusion | All | `document.json` (`doc_type="OE"`) |

### Supply Chain and Vendor Data

| Data Element | Primary step | Causal categories | Schema coverage |
|---|---|---|---|
| Vendor certifications, lot numbers, receipt inspection records | Candidates | K | `vendor_supply_chain_records.json` |
| Training records | Candidates, Conclusion | L | `training_records.json` |

## Reasoning Process: Data to Decisions

> **Note on scope revision**: scoping and data management are not strictly sequential — they are iterative. As relations and patterns are identified, the investigation boundary may need to expand (e.g., a discovered upstream cause outside the original scope). An explicit scope revision mechanism is required at each human decision point.

> **Note on uncertainty propagation**: data quality flags from step 1 must propagate and degrade confidence scores systematically through steps 3, 4, and 5. A data-limited flag is not informational only — it has a defined effect on conclusion confidence.

0. SCOPING
Define the investigation boundary before any data is examined
– Failed equipment, affected systems, trains, safety functions
– Time window: event onset + precursor horizon
– Safety function challenge assessment: which functions were lost or degraded
→ investigation scope record: equipment list, system boundary, time window, safety function map

[HD: analyst confirms scope; scope may be revised at any subsequent step]

1. DATA MANAGEMENT
Establish the two data infrastructure foundations and verify external inputs

Infrastructure initialization (runs once per plant; verified per event):
– KG: plant architecture, topology, failure modes, safety functions, document metadata — initialized from MBSE models and updated with event-specific context (equipment, time window, operating state)
– Chroma: unstructured document and narrative content — CRs, SOPs, ECAs, FMEAs, RCAs, OE documents — indexed for semantic retrieval

External inputs (provided per event; not generated by the workflow):
– Pre-processed anomalies: output of plant anomaly detection methods applied upstream to historian time-series; received as anomaly records (component, timestamp, severity, pattern); the workflow reasons over anomalies as facts — it does not perform signal processing
⚠ anomaly detection quality upstream directly limits what the workflow can conclude; gaps in anomaly coverage must be flagged
– SOE logs: millisecond-resolution discrete event sequences
– Alarm logs: filtered indicators with timestamps and priorities

Coverage check: verify KG completeness and Chroma corpus coverage for the equipment and systems in scope; flag missing or stale data
→ KG initialized and event-scoped; Chroma corpus loaded; external inputs received and coverage gaps flagged

[HD: analyst confirms data adequacy and accepts any coverage limitations before analysis proceeds]

2. KG EXPANSION
Expand the initial KG in four steps; 2a and 2b run in parallel; 2c and 2d follow after 2a and 2b complete

2a. ARCHITECTURAL SEARCH [parallel with 2b]

Expand the MBSE portion of the KG to include related events
– Identify anomalies and alarm logs associated with components and systems in the investigation scope
– Cross-reference events across types (CR, WO, alarm, anomaly, surveillance record, operator log) for the scoped equipment
→ KG expanded with event nodes and cross-references across all event types for scoped components

2b. TEMPORAL SEARCH [parallel with 2a]
For each component/asset in the KG find top-N past events
– Query KG and Chroma for historical events by component/asset
– Retrieve CRs, WOs, surveillance records, operator logs, anomalies within the precursor window and beyond
→ per-component past event lists ranked by recency and relevance

2c. TEMPORAL RELATION IDENTIFICATION [requires 2a and 2b]
Identify Allen interval relations between events
– Compute Allen relations between identified anomalies, alarm logs, and the triggering event
– Establish temporal ordering across all event types
→ Allen relation map: ordered event graph with temporal relations between all identified events

2d. SIMILAR EVENT IDENTIFICATION [requires 2b]
Identify similar events for similar equipment/components
– At plant level: similar equipment type, similar failure signature
– At fleet and industry level: OE databases (INPO, EPRI, NRC) when available
→ similar event list with provenance and confidence weight reflecting data source distance (plant > fleet > industry)

[HD: analyst reviews expanded KG, temporal relations, and similar event matches before pattern recognition proceeds]

3. PATTERN RECOGNITION – DOCUMENTARY
Retrieve past lessons learned based on patterns similar to those observed in the current event, from documentary evidence
– Match current CR, WO, and operator shift log patterns against historical CRs, WOs, RCAs, and ECAs for similar components and failure descriptions
– Each match is a lesson learned: a prior event with a known outcome, corrective action, and effectiveness result
– No match is explicitly informative: novel pattern with no

documentary precedent — candidate confidence will be lower
→ documentary lessons learned set: matched historical events with outcomes and corrective action effectiveness; novel pattern flag when no match found

[HD: analyst confirms lessons learned matches and novel pattern flags]

3.5 PATTERN RECOGNITION — — SIGNAL

Retrieve past lessons learned based on patterns similar to those observed in the current event, from signal evidence

— Match current anomalies, SOE, and alarm log patterns against historical anomaly signatures and alarm sequences for same and similar components
— Each match is a lesson learned: a prior signal pattern with a known causal explanation and resolution
— No match is explicitly informative: novel signal pattern with no historical precedent
→ signal lessons learned set: matched historical signal patterns with causal explanations; novel pattern flag when no match found

[HD: analyst confirms signal lessons learned and novel pattern flags; decides whether novel patterns indicate scope expansion]

4. CANDIDATE GENERATION AND INITIAL RANKING

Generate candidate hypotheses from sequence roots and nodes identified in steps 3 and 3.5; apply initial ranking based on local data only

Candidate definition — each candidate is a 4-tuple:
(component, failure mode, causal category, chain position)
where chain position ∈ {initiating, contributing, consequence}

Generation:

— For each sequence root and node, ask: which causal category does this pattern belong to, and what specific failure mode on this specific component best explains the observed pattern?
— Coverage enforcement: for each causal category A–L, at least one candidate must be generated or the category explicitly ruled out
— Physical plausibility gate (binary, before ranking): is this candidate physically possible given the operating state recorded in step 0? Implausible candidates are excluded with documented rationale — not scored low
— Ruling out is a named operation: every excluded candidate requires a documented reason (physically impossible / no supporting data / analyst excluded / out of scope)

⚠ Ruled-out candidates are held on standby — they receive
a second pass against fleet and industry OE evidence in step 5;
rationale is retained in output for regulatory defensibility

Initial ranking (based on local data only — no OE evidence yet):
— Temporal score: Allen relation strength and chain position
from step 2c (initiating candidates ranked above contributing)
— Logical score: topology position and support dependency
from step 2a (upstream, direct dependency ranked higher)
— Combined v1 ranking: temporal + logical scores only
→ candidate set as 4-tuples with v1 ranking;
ruled-out candidate standby list with rationale;
category coverage report

[HD: analyst reviews candidate set, v1 ranking, ruled-out list,
and coverage report; may reinstate or add candidates]

5. RANKING AND EVIDENCE ASSESSMENT
Refine ranking using documentary and OE evidence; detect unresolvable conflicts

Documentary evidence (from Chroma):
— Retrieve supporting, contradicting, and contextual snippets
per candidate from CRs, WOs, SOPs, ECAs, FMEAs, RCAs
— Consistency check: does the proposed mechanism produce
the observed event sequence? Inconsistent candidates excluded
with documented rationale

Fleet and industry OE evidence:
— Apply fleet-wide and industry OE (INPO, EPRI, NRC) as evidence
to confirm or refute active candidates
— Second pass on ruled-out standby candidates: a candidate excluded
on local data may be reinstated if strong fleet precedent exists;
reinstatement requires documented rationale
— Each OE match tagged with provenance and confidence weight
reflecting source distance (plant > fleet > industry)

Ranking refinement:
— v2 ranking: composite of v1 (temporal + logical) plus
documentary and OE evidence scores
— v1→v2 rank delta is a diagnostic: large movements indicate
evidence is discriminating between candidates
— Near-tie detection: flag candidates evidence cannot discriminate
— Sensitivity check: would ranking change if a missing data

```
   source were available?
→ hypothesis ranking v2; evidence posture per candidate;
   reinstated candidates with rationale; near-tie flags;
   sensitivity table; unresolvable gaps flagged

      [HD: analyst reviews v2 ranking, evidence posture, reinstated
       candidates; confirms or overrides before conclusion]

6. CONCLUSION
   Draw and document the causal conclusion
   – Primary hypothesis: proximate, contributing, and root cause levels
   – Barrier analysis: which barriers failed, which held, and why
   – Human performance assessment
   – Unresolved gaps: what evidence would change the conclusion
   – Recommended actions per causal level
   – Forward-looking: what indicators should be monitored
     to verify corrective action effectiveness
   → RCA card (CAP-ready): primary conclusion, alternatives,
     evidence linkage, barrier assessment, recommended actions,
     effectiveness monitoring plan, open gaps

      [HD: analyst approves conclusion and signs off on AP-913 checklist]
```

## Causal Depth Levels

The categories have a natural causal depth structure relevant to AP-913 and 10 CFR 50 Appendix B:

| Level | Categories | AP-913 term |
|---|---|---|
| Proximate cause | A, B, C, D, E, F | Direct cause — immediate physical mechanism |
| Contributing cause | G, H, I, J, K | Factors that allowed the proximate cause to exist |
| Root cause | L | Systemic weakness that allowed contributing causes to persist |

The current pipeline reasons almost entirely at the proximate cause level. A complete nuclear RCA requires traversing all three levels. Recommended actions addressing only the proximate cause (e.g.,

"replace the failed bearing") without the contributing cause (e.g., "PM interval inadequate") and root cause (e.g., "AMP not updated to reflect fleet OE") will not satisfy regulatory expectations.

## Candidates vs. Causal Categories

**Causal categories (A–L)** are classes of causal mechanisms — they define the *type* of cause and ensure investigation coverage. Coverage enforcement operates at the category level: did the pipeline generate at least one candidate in each applicable category?

**A candidate** is a specific, instance-level hypothesis: a particular failure mode, on a particular component, at a particular time, attributed to a primary causal category. Ranking and evidence assessment operate at the candidate level.

Example:

- Category A = "equipment-internal degradation" (class)
- Candidate = "bearing wear on Pump P-101A, failure mode FM-047, consistent with the trip at 14:32" (instance)

One category can generate multiple candidates. One candidate belongs to exactly one primary category.

## Coverage Requirement

For each event, the pipeline must either:

1. Generate and score at least one candidate in each category, **or**
2. Explicitly document in `rca_card.executive_summary.analyst_attention_flags[]` why the category was ruled out or is not applicable.

# Step 5 — Ranking and Evidence Assessment Strategy

Step 5 combines hard constraint elimination with evidence posture classification. Applied sequentially — elimination first, then posture classification and ranking on surviving candidates.

# Phase 1 — Hard Constraint Elimination

Apply binary gates to eliminate physically or logically impossible candidates before any scoring. These are facts, not evidence. Eliminated candidates go to the ruled-out log with documented reason and held on standby for OE second pass.

**Gate 1 — Physical Plausibility**
Is this failure mode possible given the operating state at event time?

- Operating state record (step 0): power level, flow rates, pressures, temperatures, operating mode
- FMEA failure mode parameters: operating conditions under which each failure mode is possible
- Design basis documents: operating envelope limits per component
- Equipment specifications: rated conditions, material properties

**Gate 2 — Timeline Consistency**
Does the proposed mechanism produce the observed event sequence?

- When FMEA latency parameters are available: hard gate — observed lag outside expected latency window → candidate eliminated
- When FMEA latency parameters are absent (the common case): degrades to Allen relation check only:
  - Anomaly FOLLOWS event → eliminated as consequence not cause
  - Anomaly PRECEDES or OVERLAPS → passes to Phase 2 as soft temporal signal
  - ⚠️ FMEA latency parameters are rarely available — timeline consistency will most often operate in degraded mode; temporal discrimination burden shifts to Phase 2
- Data: Allen relation map (step 2c), SOE log, FMEA latency parameters (when available), confirmed event timeline

**Gate 3 — Barrier Logic**
If a barrier held, candidates requiring that barrier to fail are eliminated.

- Safety function definitions from KG: which barriers were active at event time
- Protection logic: which barriers held and which failed (SOE actuation records)
- Equipment status: barrier availability, test status, operability
- Alarm logs: confirmation of barrier actuation or non-actuation
- ⚠️ Requires protection logic to be modeled in the KG — significant data requirement

→ Surviving candidate set; ruled-out log with gate and reason; standby list for OE second pass

# Phase 2 — Evidence Posture Classification

For each surviving candidate, classify evidence posture independently across four streams:

| Stream | Source | Posture basis |
|---|---|---|
| Temporal | Allen relation map (step 2c) | Strength and consistency of temporal precedence |
| Logical | KG topology (step 2a) | Upstream position, support dependency, proximity |
| Documentary | Chroma retrieval, lessons learned (steps 3, 5) | Supporting, contradicting, contextual snippets |
| OE | Fleet and industry matches (steps 2d, 5) | Prior events with same pattern and known outcome |

Each stream independently returns: `supported | contradicted | mixed | insufficient`

→ Per-candidate evidence posture across four streams

# Phase 3 — Posture Aggregation and Ranking

**Aggregation rules**:

- Contradicted by any single stream → cannot be primary; flagged for analyst review
- Supported by all four streams → strongest possible conclusion
- Mixed or insufficient streams → confidence level and analyst review requirements set accordingly

**Ranking**:

- Within posture classes: fully supported > partially supported > mixed > insufficient
- Within each class, number of supporting streams breaks ties
- Strong temporal and logical support but insufficient documentary → ranks below candidate supported across all four streams

**Confidence and review flags**:

- Near-tie between top two candidates → analyst review required
- Primary candidate with any contradicted stream → analyst review required
- Sensitivity check: would ranking change if a missing data source were available?

→ Hypothesis ranking v2; candidate-level posture; confidence level; near-tie flags; sensitivity table; unresolvable gaps flagged

# Implementation Status (2026-04-25 EOD)

Step-level readiness against the Step 0-6 definitions above:

- **Step 0 — Scoping: Green**
  - Implemented: baseline scope capture plus versioned iterative scope-revision lifecycle in `run_context` (trigger, boundary delta, analyst decision, timestamp, active approved scope version/revision id), with strict consistency checks and manifest-level scope runtime summary for auditability.
  - Residual gap: stage-specific revision triggers from Step 2/3.5 not yet fully auto-wired (handled in `phase2_scope_expansion_hooks_plan_april_25.md`).
- **Step 1 — Data Management: Green**
  - Implemented (2026-04-25): 8-family coverage report (`kg_context`, `chroma_corpus`, `upstream_anomaly_inputs`, `telemetry_detail`, `soe_log`, `alarm_log`, `protection_logic_context`, `configuration_change_records`); per-artifact quality field consumption; `not_assessed` status for absent optional families; paired-data coupling check (SOE ↔ protection logic context) surfaced in `review_hooks.degraded_reasons`; weighted coverage quality factor in scoring engine; full-mode strict-mode validation (missing families, telemetry mandatory, paired-data requirement, overall-status consistency, `paired_data_checks` block required); 24 targeted tests + full suite 787 passed.
- **Step 2 — KG Expansion: Green (2b + 2c)**
  - Implemented (2b, 2026-04-25): temporal search post-processing — per-event `in_precursor_window` / `window_tier` tagging (configurable precursor_window_days, default 180); `per_component_past_events` index with per-component top-N cap; `temporal_search_summary` in `seed_context` and `run_manifest.pipeline_config.temporal_search`; 20 targeted tests.
  - Implemented (2c, 2026-04-25): Allen relation map computed for anomalies, alarm entries, and SOE records against the triggering event — `_build_allen_relation_map` static method; clock-sync failure → forced `"unknown"` relation; large SOE logs capped at `max_soe_nodes`; `allen_relation_map.json` schema created; artifact wired into run

manifest top-level and `artifacts` block; 24 targeted tests — all pass; total suite 831 passed.
    - The KG query ( `_fetch_past_events` ) and CMMS live supplement were already in place.
    - Residual gaps: Step 2a (Architectural Search), Step 2d (fleet/industry OE similar events) — deferred. Allen base scores not yet forwarded into causality scoring pipeline (Step 4 work).
- **Step 3 / 3.5 — Pattern Recognition (Documentary and Signal): Green**
    - Implemented (2026-04-25): `novel_pattern` boolean on every `tskr_patterns.patterns[]` entry (True when `recurrence_count == 0` , history score < 0.20, and no signal IDs). `n_novel_patterns` / `has_novel_patterns` added to `tskr_patterns.summary` .
    - `alarm_log` and `soe_log` threaded into `TSKRTemporalScorerV1.score()` via new `_extract_alarm_windows` and `_extract_soe_windows` static methods; alarm/SOE point-event windows merged into the anomaly pool for pattern scoring; clock-sync failures mark windows as degraded without crashing. Backward-compatible via `inspect.signature` guard in `_build_tskr_patterns` .
    - `_build_signal_lessons_learned` static method builds the Step-3.5 artifact: separates `matched_patterns` (recurrence > 0 or history ≥ threshold) from `novel_patterns` ; attaches `causal_explanation` and `resolution_summary` from recurrence profile; populates `input_sources` list.
    - `signal_lessons_learned.json` schema created.
    - `signal_lessons_learned` wired into run manifest top-level and `artifacts.signal_lessons_learned` (summary only).
    - Novel-pattern outcomes already wired to scope-revision suggestions via Phase 3b.
    - 19 tests in `test_step35_signal_lessons_learned.py` — all pass; total suite 867 passed.
- **Step 4 — Candidate Generation and Initial Ranking: Green**
    - Implemented: canonical candidate 4-tuple, A-L coverage/rule-out enforcement, and elimination-first hard-gate auditable behavior.
- **Step 5 — Ranking and Evidence Assessment: Green**
    - Implemented: elimination-first gates, per-stream posture, contradiction blocking, near-tie/sensitivity outputs, OE reinstatement with rationale/provenance, replayability signature for deterministic comparisons, and **sensitivity table** (2026-04-25).
    - Sensitivity table: for each top-N candidate, projects composite-score delta per missing/degraded data source; injects analyst attention flag when ranking change is possible; 25 dedicated tests.
- **Step 6 — Conclusion: Green**
    - Implemented (2026-04-25): depth-complete RCA card (proximate/contributing/root), depth-mapped recommended actions, `depth_incomplete_reason` explaining unresolved layers, deepened `unresolved_gaps` (contributing/root layer gaps, sensitivity table link, novel pattern flag), depth-stratified `effectiveness_monitoring_plan`

(equipment/process/programmatic indicators with `success_criteria`), and `human_performance_assessment` block (H/I/J/K category findings, performance mode mapping, AP-913 regulatory references, corrective action cross-linking).
    - 41 targeted tests; 933 total pass, zero regressions.

## What Remains For Strong Metamodel-Complete State

1. Propagate scope-revision decisions from Steps 2 and 3.5 into downstream candidate/ranking stages. **Completed 2026-04-25** — see Scope-Revision Downstream Propagation section in backlog.
2. Tighten Step 1 source-coupling quality semantics where the metamodel expects hard paired data availability. **Completed 2026-04-25** — `soe_plc_pairing` renamed to `"violated"`; escalated to `analyst_decisions_required`.
3. Step 2d (fleet/industry similar events) — **Completed 2026-04-25**.
4. Step 2a (architectural search) — deferred.

## Execution Sequencing (What and How)

- **Phase 1 (completed): Step 0 iterative scope revision mechanism**
    - Added versioned scope records with trigger, boundary delta, analyst decision, and timestamp.
    - Added active approved scope version/revision references on `run_context.input_refs` for downstream stage consumption.
    - Added strict semantic checks and unit tests for revision lifecycle integrity.
- **Phase 2 (completed 2026-04-25): Harden Step 1 — Data Management**
    - Expanded coverage report to 8 source families with per-artifact quality.
    - Added paired-data coupling check and weighted quality factor.
    - Added full-mode strict validation and 24 targeted tests.
    - Detailed plan: `step1_data_management_hardening_plan_april_25.md`.
- **Phase 3a (completed 2026-04-25): Harden Steps 2b + 2c — Temporal Search and Allen Relation Map**
    - Step 2b: per-event window tagging, per-component indexing with top-N cap, `temporal_search_summary` in manifest; 20 tests.
    - Step 2c: Allen relation map computed for anomalies, alarms, SOE records — `_build_allen_relation_map`; clock-sync safety; `allen_relation_map` schema + manifest wiring; 24 tests.
    - Detailed plans: `step2b_temporal_search_hardening_plan_april_25.md`, `step2c_allen_relation_map_plan_april_25.md`.

- **Phase 3b (completed 2026-04-25): Formalize scope-expansion hooks in Steps 2 and 3.5**
  - `_detect_scope_expansion_signals` static method scans Allen relation map (out-of-scope causal components), signal evidence propagation chains (out-of-scope components), and TSKR patterns (novel/no-match) to emit typed `ScopeExpansionSignal` dicts.
  - `_inject_scope_expansion_signals` merges signals into `run_context.scope_management.expansion_suggestions` (idempotent on `signal_id`).
  - Called in `run()` after Step 2c map is built and before `_stage_g_finalize_manifest`.
  - `scope_expansion_summary` (total, pending, by_trigger_type) propagated to manifest top level.
  - `review_hooks.analyst_decisions_required` and `degraded_reasons` populated when pending signals exist.
  - `run_context.json` schema updated with `expansion_suggestions` array in `scope_management`.
  - `similar_event_list.json` schema created; stub artifact (`status: not_implemented`) wired into manifest for Step 2d contract stability.
  - 17 tests in `test_step3b_scope_expansion_hooks.py` — all pass; total suite 848 passed.
- **Phase 4: Step 6 conclusion maturity gaps (completed 2026-04-25)**
  - Added `human_performance_assessment` block (H/I/J/K findings, performance mode, AP-913 refs, corrective-action cross-links).
  - Deepened `unresolved_gaps` with contributing/root layer gaps, sensitivity table link, novel pattern flag.
  - Depth-stratified `effectiveness_monitoring_plan` (proximate/contributing/root profiles, `success_criteria`).
  - Added `depth_incomplete_reason` to `causal_depth_summary` when depth_complete=false.
  - 41 targeted tests in `test_step6_conclusion.py`; 933 total tests pass.
- **Phase 5: Full workflow logic audit (completed 2026-04-25)**
  - Audited data-flow integrity, schema contracts, step ordering, quality-flag propagation, hard gates, and gap-builder input wiring.
  - **Fixed A:** `_build_unresolved_gaps` was reading `sensitivity_any_change` from `run_context["sensitivity_table"]` (never set) and `novel_pattern_flag` from `run_context["tskr_patterns"]` (never set). Now reads from `causality_candidates["sensitivity_table"]` and the `tskr_patterns` parameter respectively. `_fallback_card` accepts `tskr_patterns` as a new optional parameter.
  - **Fixed E:** `sensitivity_table.json` `summary.top_n_candidates` had `minimum: 1` (schema violation for empty-candidate runs); corrected to `minimum: 0`.
  - **Fixed F:** `_build_signal_lessons_learned` coerces `pattern_id` and `confidence` to non-null values (string/float fallback) to prevent schema violations when TSKR rows are incomplete.

- **Accepted-by-design:** Phase 3b scope-expansion signals are produced after the card build and are intended as next-run analyst inputs, not same-run card feedback. `allen_relation_map` is manifest-level (not synthesizer-level) by design. Step 1 `coverage_summary` does not feed `generate()` by design (quality penalty applied in `refine_with_evidence` only).
- 933 tests pass after fixes, zero regressions.

- **Phase 5 — Finding 4 (completed 2026-04-25): Surface Categories F/K/L data availability to analyst**
  - Categories F (environmental monitoring), K (vendor/supply-chain records), and L (training records) had dedicated JSON schemas but were invisible to the coverage report.
  - `run()` now accepts `environmental_monitoring`, `vendor_supply_chain_records`, and `training_records` as optional `JsonDict` parameters (checked against `input_refs` flags as fallback).
  - `_build_data_coverage_summary` assesses each family and assigns `not_assessed` / `complete` / `partial` status using field-level quality heuristics.
  - `_stage_g_finalize_manifest` signature and its call site in `run()` updated to thread the new parameters through.
  - Validator `all_expected_families` set expanded to include the three new families (required in full mode; `not_assessed` is valid).
  - `ALL_EXPECTED_FAMILIES` constant in `test_step1_data_coverage.py` updated; 24 existing Step 1 tests still pass; full suite 933 passes, zero regressions.
- **Phase 5 — Finding G (completed 2026-04-25): Wire `allen_base_score` into causality engine scoring**
  - Step 2c Allen relation map nodes carry `allen_base_score` (0–1) and `allen_relation_to_event`, but these were consumed only at manifest level — `refine_with_evidence` (called before `_stage_g_finalize_manifest`) never saw them.
  - **Sequencing fix:** `_build_allen_relation_map` hoisted out of `_stage_g_finalize_manifest` to `run()`, before `refine_with_evidence`. Pre-computed result is reused in `_detect_scope_expansion_signals` and passed to `_stage_g_finalize_manifest` as `pre_computed_allen_map` (no rebuild).
  - **New static helpers in `causality_engine_v32.py`:**
    - `_build_allen_component_index`: indexes causal nodes by `component_id`; applies SOE clock-sync discount (×0.80); builds `follow_ids` set.
    - `_apply_allen_temporal_blend`: blend `new_temporal = 0.75×TSKR + 0.25×allen` (causal match only; can only raise, not lower); updates `composite_raw` / `composite_score` via temporal-weight delta; sets `temporal_evidence["temporal_contradiction"]=True` for `follows` nodes.

- **Automatic timeline gate:** contradiction flag is read by the existing `_apply_timeline_consistency_gate`, which rules out the candidate with `ruleout.reason_code="timeline_inconsistent"`.
- New score fields: `scores["allen_temporal_score"]`, `scores["allen_relation"]`, `scores["allen_blend_applied"]`; blend note appended to `score_rationale["temporal"]`.
- `refine_with_evidence` accepts `allen_relation_map: Optional[JsonDict] = None` — fully backward-compatible.
- 25 tests in `test_finding_g_allen_scoring.py`; **958 total tests pass, zero regressions**.

- **Phase 5 — Finding I (completed 2026-04-25): Direct `protection_logic_context` read in hard gates**
  - Both protection hard gates (physical plausibility, barrier logic) operated solely on structural proxies; `protection_logic_context` (`barrier_states`, `logic_sets`) was threaded through the pipeline but never consulted by the gates themselves.
  - **New static helper `_build_plc_barrier_index`** in `causality_engine_v32.py`: parses `barrier_states[]` into `{sf_id → state}` and flattens all `logic_sets[].input_signals` / `output_signals` into a `set[str]`. Built once per `refine_with_evidence` call.
  - **`_apply_physical_plausibility_gate`** enhanced: when `component_id ∈ plc_logic_signal_ids` → sets `plc_consulted=True`; if any `affected_safety_functions[].sf_id` has `state="held"` → protection-system-responded note added to rationale (gate still passes — informational).
  - **`_apply_barrier_logic_gate`** enhanced: when any matched `sf_id` has `state ∈ {failed, degraded}` → gate FAILS (`plc_forced_fail=True`, `ruleout.reason_code="barrier_held"`); `state="held"` appended to rationale only; `plc_consulted` flag recorded.
  - **`refine_with_evidence`** accepts new param `protection_logic_context: Optional[JsonDict] = None` (fully backward-compatible); PLC index built once; passed to both gates.
  - **Orchestrator** updated: `refine_kwargs` includes `protection_logic_context` via the existing inspect-guard pattern; `_run_auto_reentry_if_needed` signature and internal refine call updated.
  - 22 tests in `test_finding_i_plc_gates.py`; **980 total tests pass, zero regressions**.
- **Phase 5 — Step 2d (completed 2026-04-25): Similar Event Identification — three-tier OE lookup**
  - Plant tier: `_query_plant_past_events` static method scores `kg_context.past_events` on 5 match dimensions (component +0.40, failure-mode +0.25, event-type +0.15, actuation

+0.10, precursor-window +0.10); top-N returned sorted by confidence_weight; tier multiplier 1.0.
    - Fleet and industry tiers: `SimilarEventAdapter` Protocol ( `runtime_checkable` ) injected via `set_similar_event_adapter()` ; concrete `LLMOEAdapter` builds structured prompt and POSTs to fine-tuned endpoints (INPO/EPRI/NRC); timeout and HTTP errors → `degraded_tiers` list, no crash; tier multipliers 0.80 (fleet) / 0.60 (industry).
    - `_annotate_candidates_with_oe_evidence` : mutates top candidates in-place; injects matched events (threshold ≥ 0.30) into `oe_reinstatement_evidence` keyed by `component_id` or `failure_mode_id` .
    - `_build_unresolved_gaps` updated: emits gap when `plant_count=0` ; emits gap per degraded tier.
    - `synthesize()` and `_fallback_card()` accept `similar_event_list` ; built before `synthesize()` in `run()` to feed the card.
    - `similar_event_list.json` schema upgraded with `query_terms` , `summary` , `match_dimensions` , OE-specific event fields.
    - 23 tests in `test_step2d_similar_events.py` ; **1003 total tests pass, zero regressions**.
- **Phase 5 — Finding H (completed 2026-04-25): Category E `operating_point` in Causal Scoring**
    - New `_operating_point_score` static helper: 7-mode base table ( `power_ramp` =0.70 → `shutdown` =0.20); Cat E power modifier (high-demand keywords `× 0.30 × p_norm` ; standby keywords `× 0.25 × (1−p_norm)` ); train OOS + standby bonus `+0.15` ; returns `(0.0, "not_assessed")` when context absent.
    - `_build_failure_mode_candidates` reordered: `_infer_primary_category_for_failure_mode` now called *before* structural assembly so `op_delta` can use the correct category (sequencing fix).
    - Operating-point delta `op_delta = 0.12 × op_score` (cap +0.12) added to structural sum; `scores["operating_point_score"]` and `scores["operating_point_note"]` stored on every candidate.
    - `score_rationale["structural"]` (initial and post-refinement) includes operating-point note when `op_score > 0` .
    - 20 tests in `test_finding_h_operating_point.py` ; **1023 total tests pass, zero regressions**.
- **Phase 5 — Scope-Revision Downstream Propagation (completed 2026-04-25)**
    - Two new static helpers in `rca_reasoning_orchestrator.py` :
        - `_resolve_approved_scope_boundary` : returns normalised frozenset of approved `component_ids` when `active_scope_version > 0` ; `None` in discovery mode (v0) or empty boundary.

        - `_apply_scope_boundary_filter` : soft-filters candidates — out-of-scope `component_id` s moved to `ruled_out[]` with `reason_code="scope_filtered"` , `hard_gate=False` ; meta-fields `scope_filter_applied/version/filtered_count/filtered_component_ids` stored.
    - `apply_scope_revision` enhanced: auto-builds `scope_snapshot` by merging `changed_boundary.added/removed_component_ids` into the prior accepted snapshot when caller omits `scope_snapshot` .
    - New `resolve_expansion_suggestion` instance method: atomic bridge between expansion-suggestion write path and scope-revision lifecycle — marks `analyst_decision` , stores `resolution_timestamp` / `analyst_rationale` , and (when accepted) calls `apply_scope_revision` with `suggested_component_ids` .
    - Filter wired into `run()` between `generate()` and `refine_with_evidence` ; `scope_filter` block surfaced in `run_manifest.artifacts` .
    - `causality_candidates.json` `reason_code` enum extended with `"scope_filtered"` .
    - 26 tests in `test_scope_revision_downstream.py` ; **1049 total tests pass, zero regressions**.
- **Phase 5 — Item 2 (completed 2026-04-25): Step 1 SOE/PLC paired-data hardening**
    - `soe_plc_pairing` value changed from `"warning"` → `"violated"` when SOE is present but PLC is absent (clearer severity vocabulary).
    - `_compute_review_hooks` now appends a structured entry to `analyst_decisions_required` (not just `degraded_reasons` ) when pairing is `"violated"` or `"warning"` , requiring explicit analyst acknowledgement before writeback.
    - Validator condition broadened to `plc_s in {"missing", "violated"}` (future-proofing).
    - 3 existing Step-1 tests updated; 1 new assertion on `analyst_decisions_required` . Zero regressions.
- **Phase 5 — Item 3 (completed 2026-04-25): Category C CCF structural delta**
    - `_build_failure_mode_candidates` pre-computes CCF features using a mini-candidate `{cause_node_id, kg_path}` before structural assembly (avoids double call).
    - `ccf_delta = 0.10 × common_cause_score` added to structural sum **only when `primary_causal_category == "C"`** (max +0.10 advisory contribution).
    - `scores["ccf_score"]` and `scores["ccf_note"]` stored on every candidate.
    - Initial `score_rationale["structural"]` includes CCF note when active; `_update_score_rationale_for_refinement` appends CCF note post-refinement.
    - Pre-computed `pre_ccf` dict reused for `candidate["common_cause"]` — eliminates redundant second call.
    - Zero regressions; **1049 total tests pass**.